

Agentic AI with LLMs

From Frozen Models to Autonomous Systems

Eric Chen
2026-5-6



LLMs Are Frozen in Time

Problems with LLMs

- ❖ LLM weights = *compressed snapshot* of training data
- ❖ **Knowledge cutoff**: typically months to years old
- ❖ **No native access** to:
 - Private data
 - Real-time information
 - Anything generated after training
- ❖ **Heavy hallucination** on critical knowledge & reasoning

Core Idea: Allow LLMs to **access information** outside of its own pretrained weights

- ❖ **First Step:** Retrieval Augmented Generation (RAG)

Outdated Facts

Q: Where is the winter olympic this year?

A: Beijing China

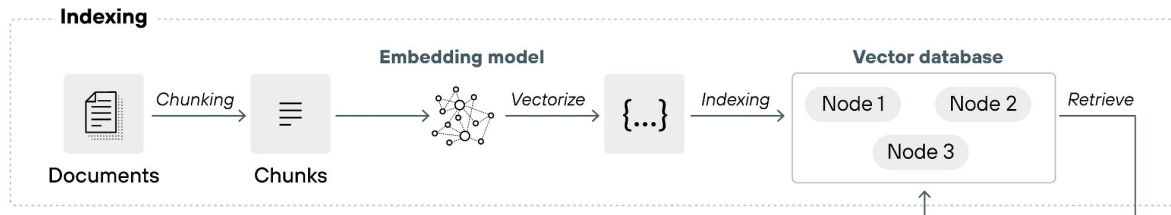
Fiction

Q: What rules do all artificial intelligences currently follow?

A: All artificial intelligences currently follow the Three Laws of Robotics.

RAG: Retrieval Augmented Generation

RAG architecture

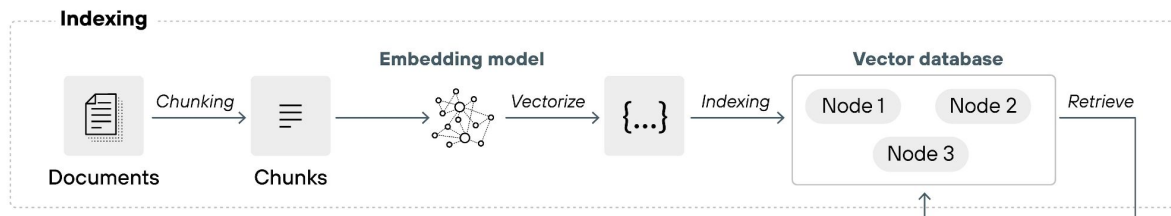


Building Retrieval Material:

- ❏ **Ingest** — load documents (PDFs, HTML, DBs, etc.)

RAG: Retrieval Augmented Generation

RAG architecture

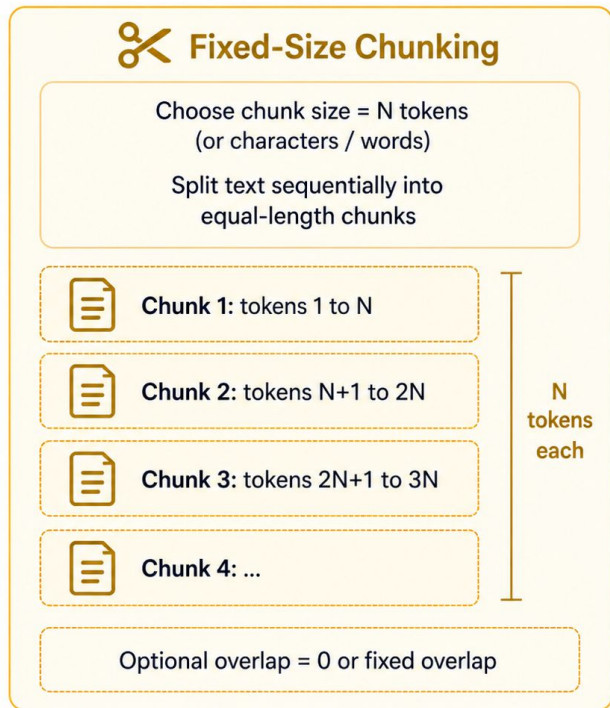


Building Retrieval Material:

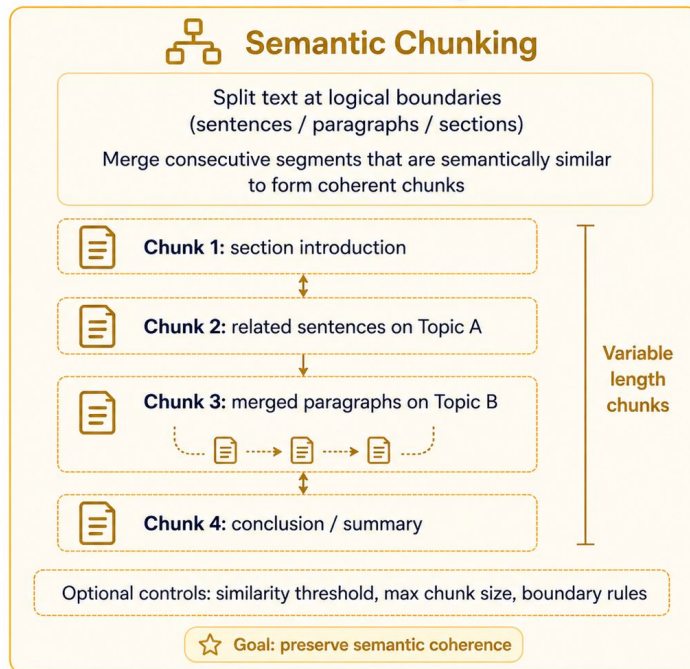
- ❑ **Ingest** — load documents (PDFs, HTML, DBs, etc.)
- ❑ **Chunk** — split into semantic units (e.g., 200–800 tokens)

RAG: Retrieval Augmented Generation

Fixed-Size Chunking for RAG

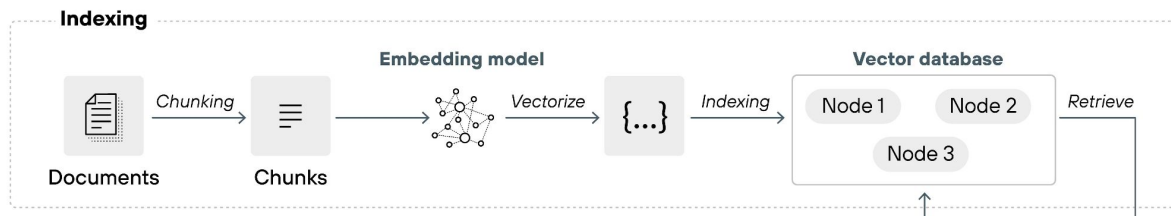


Semantic Chunking for RAG



RAG: Retrieval Augmented Generation

RAG architecture

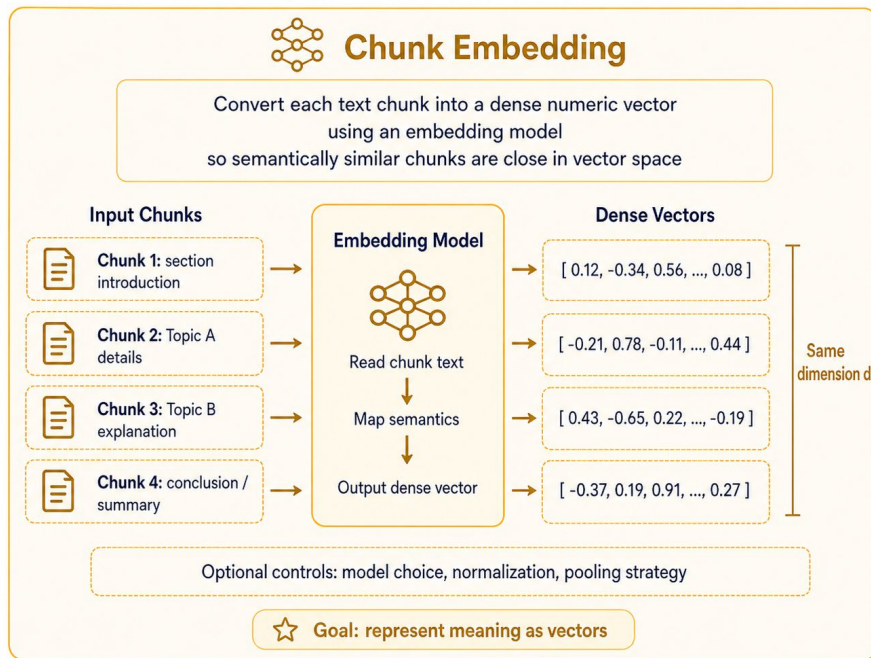


Building Retrieval Material:

- ❑ **Ingest** — load documents (PDFs, HTML, DBs, etc.)
- ❑ **Chunk** — split into semantic units (e.g., 200–800 tokens)
- ❑ **Embed** — encode chunks as dense vectors

RAG: Retrieval Augmented Generation

Embedding Chunks into Dense Vectors



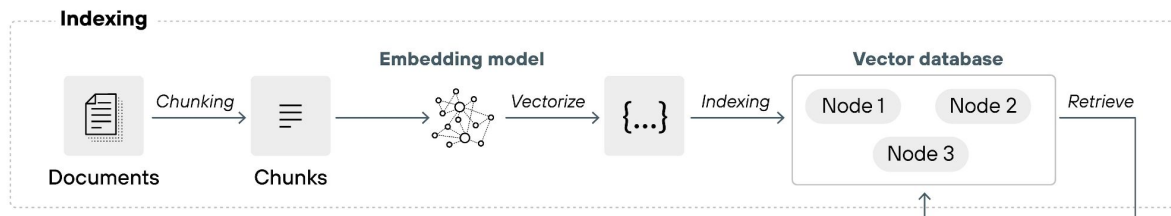
Common embedding models:

- ❖ SentenceTransformer, OpenAI text-embedding, Cohere embed-v4, Voyage



RAG: Retrieval Augmented Generation

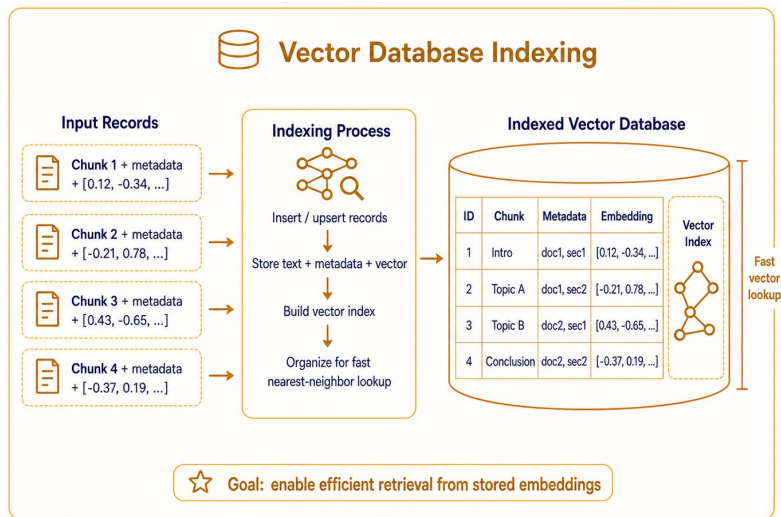
RAG architecture



Building Retrieval Material:

- ❑ **Ingest** — load documents (PDFs, HTML, DBs, etc.)
- ❑ **Chunk** — split into semantic units (e.g., 200–800 tokens)
- ❑ **Embed** — encode chunks as dense vectors
- ❑ **Index** — store in a vector DB

RAG: Retrieval Augmented Generation



❖ **Indexing:** organizes high-dim embeddings s.t. Fast retrieval of similar vectors using ANN search

❖ **Common Index Types:**

- *HNSW*: graph-based, good recall + low latency.
- *PQ (Product Quantization)*: compresses vectors into compact codes, reduce memory.
- LSH, IVF, Brute-force

❖ **Key Trade-offs:** speed vs. accuracy (recall), memory footprint, and index build time.

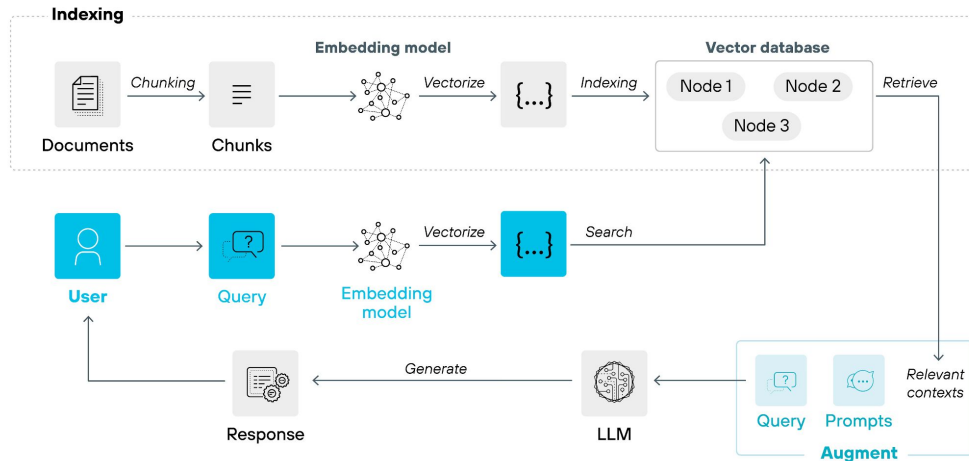
❖ **Common Vector DBs:**

- Pinecone, Chroma, FAISS, pgvector



RAG: Retrieval Augmented Generation

RAG architecture



Retrieval-Augmented Generation (RAG)

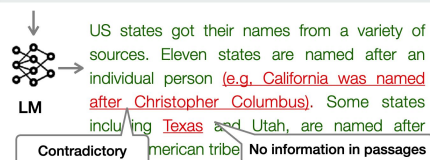
Prompt How did US states get their names?

Step 1: Retrieve K documents

- 1 Of the fifty states, eleven are named after an individual person.
- 2 Popular names by states. In Texas, Emma is a popular baby name.
- 3 California was named after a fictional island in a Spanish book.

Step 2: Prompt LM with K docs and generate

Prompt How did US states get their names? + 1 2 3



Retrieval then Generate:

- ❑ **Retrieve** — at query time, embed the query, retrieve the top-k chunks (optionally re-rank)

RAG: Retrieval Augmented Generation

❖ Retrieve Search:

- **Query Embedding:** Encoded with the same indexing embedder.
- **Similarity Search:** fetch top-k candidates from DB; ANN.
- **Distance Metrics:** cosine similarity, dot product, & L2 distance.
- **Hybrid search:** sparse (BM25) and dense retrieval to improve recall.

❖ Rerank:

- **Re-scores** the top-k chunks using a more powerful cross-encoder model.
- **Cross-encoder:** Jointly attends to query and chunk for fine-grained relevance scoring.
- **Improves precision, reduces noise**
- **Common Re-rankers:** Cohere Rerank, [BGE-reranker](#), [Jina Reranker](#)

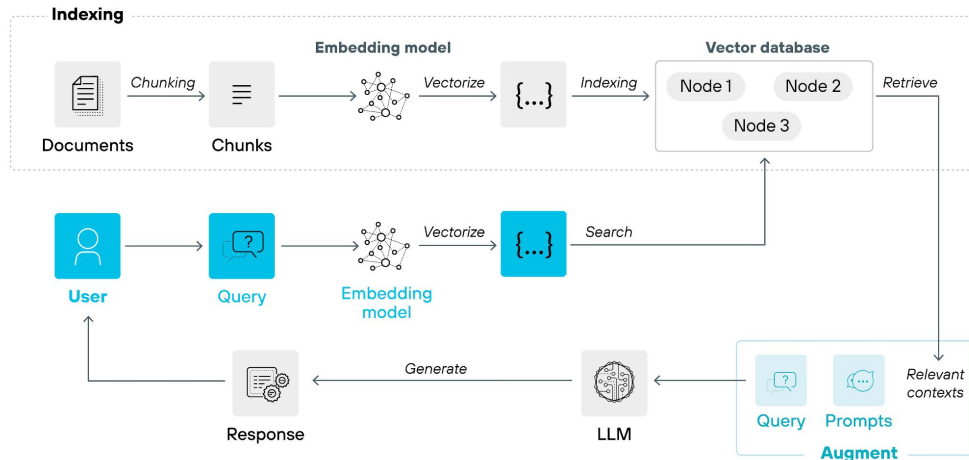


BGE



RAG: Retrieval Augmented Generation

RAG architecture



Retrieval-Augmented Generation (RAG)

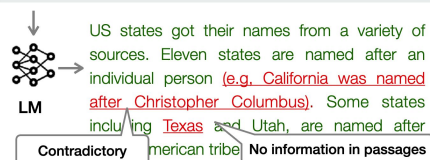
Prompt How did US states get their names?

Step 1: Retrieve K documents

- 1 Of the fifty states, eleven are named after an individual person.
- 2 Popular names by states. In Texas, Emma is a popular baby name.
- 3 California was named after a fictional island in a Spanish book.

Step 2: Prompt LM with K docs and generate

Prompt How did US states get their names? + 1 2 3



Retrieval then Generate:

- ❑ **Retrieve** — at query time, embed the query, retrieve the top-k chunks (optionally re-rank)
- ❑ **Generate** — concatenate top-k chunks to the prompt; LLM generate a response

RAG: Retrieval Augmented Generation

Building Blocks of a RAG Pipeline

- ❖ **Pipeline:** LangChain, LlamaIndex, DSPy
- ❖ **Vector DBs:** Pinecone, Chroma, FAISS, pgvector
- ❖ **Embeddings:** SentenceTransformer OpenAI text-embedding, Cohere embed-v4, Voyage
- ❖ **Re-rankers:** Cohere Rerank, [BGE-reranker](#), [Jina Reranker](#)



Pinecone



Chroma



FAISS



Pgvector



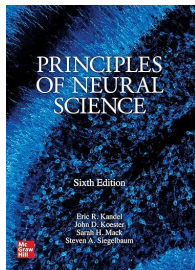
BGE



Where RAG Falls?

Limitations of RAG

- ❖ **Single-shot retrieval:**
 - No refinement if the first hit is irrelevant
- ❖ **Not ideal for dynamic information:**
 - Static: Requires reindexing when embedder/chunking/document changes
- ❖ **Context Window Bloat:**
 - Top-k chunks burn tokens & fill the context window



arXiv

PubMed



SEMANTIC SCHOLAR



National
Center for
Biotechnology
Information

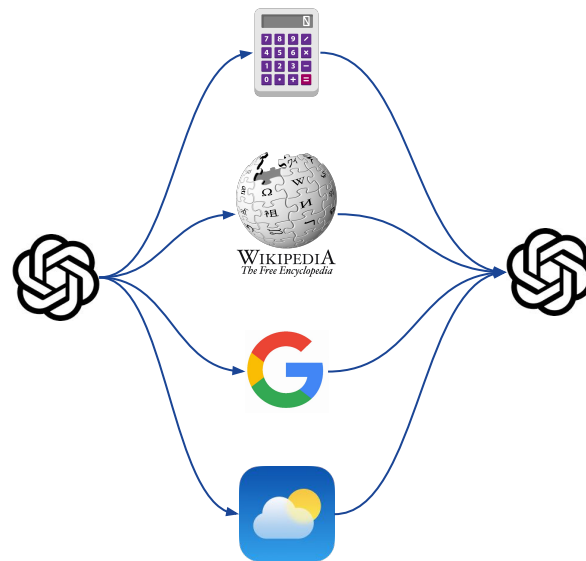
LLM with Tools

The New England Journal of Medicine is a registered trademark of [QA("Who is the publisher of The New England Journal of Medicine?") → Massachusetts Medical Society] the MMS.

Out of 1400 participants, 400 (or [Calculator(400 / 1400) → 0.29] 29%) passed the test.

The name derives from "la tortuga", the Spanish word for [MT("tortuga") → turtle] turtle.

The Brown Act is California's law [WikiSearch("Brown Act") → The Ralph M. Brown Act is an act of the California State Legislature that guarantees the public's right to attend and participate in meetings of local legislative bodies.] that requires legislative bodies, like city councils, to hold their meetings open to the public.



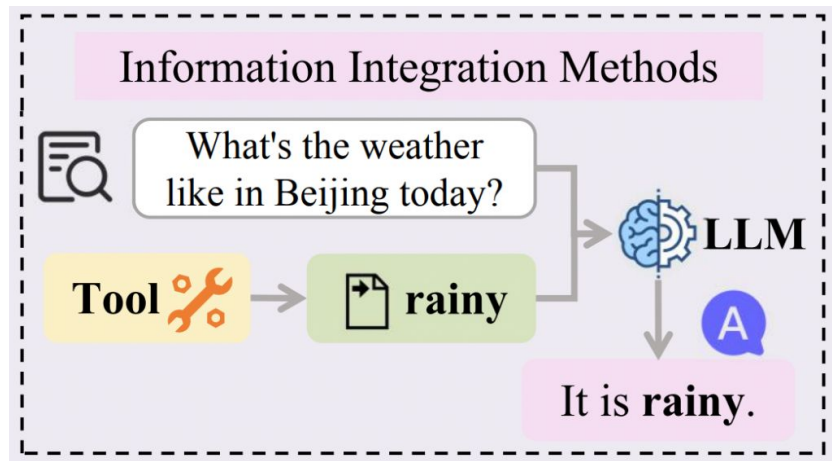
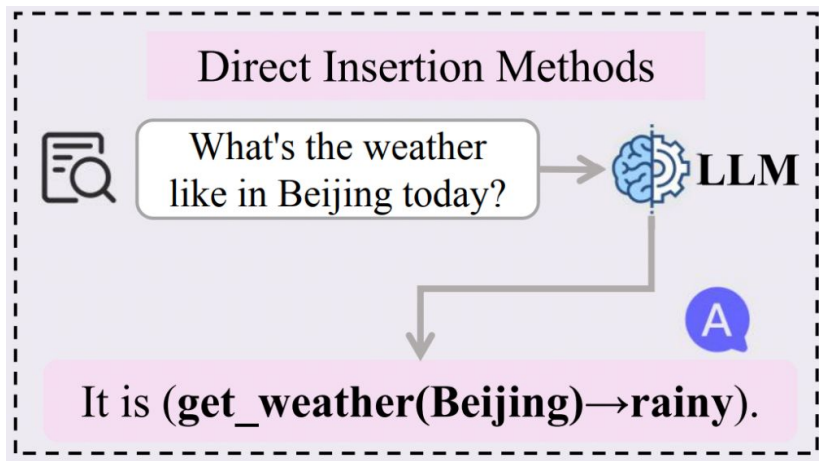
Toolformer

Train LLM to make tool calls

LLM with Tools

1. Direct insert the tool's output into the generation
 - Less flexibility,
 - Less hallucination

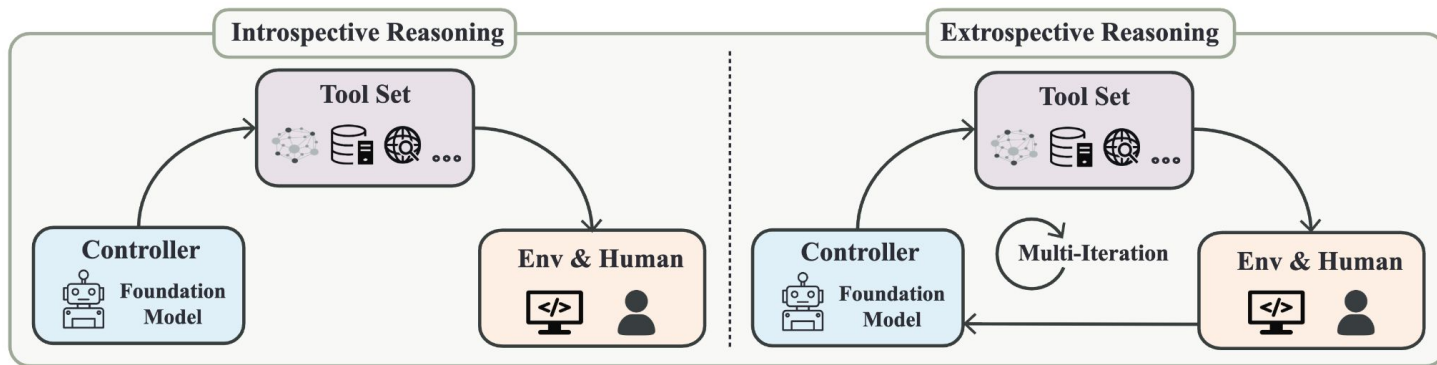
2. LLM generate response based on tool's output
 - More flexibility
 - More hallucination



LLM with Tools

	Bring info to the model	Let the model fetch info
Mechanism	Adding retrieved docs to the context	Give model tools to call
Best for	<i>Static, well-bounded knowledge</i>	<i>Dynamic, long-tail, multi-step tasks</i>
Pattern	RAG	Tool use

Learning to Call Tools



Learning from Demonstrations:
learning from human-annotated or
model-annotated demonstrations;

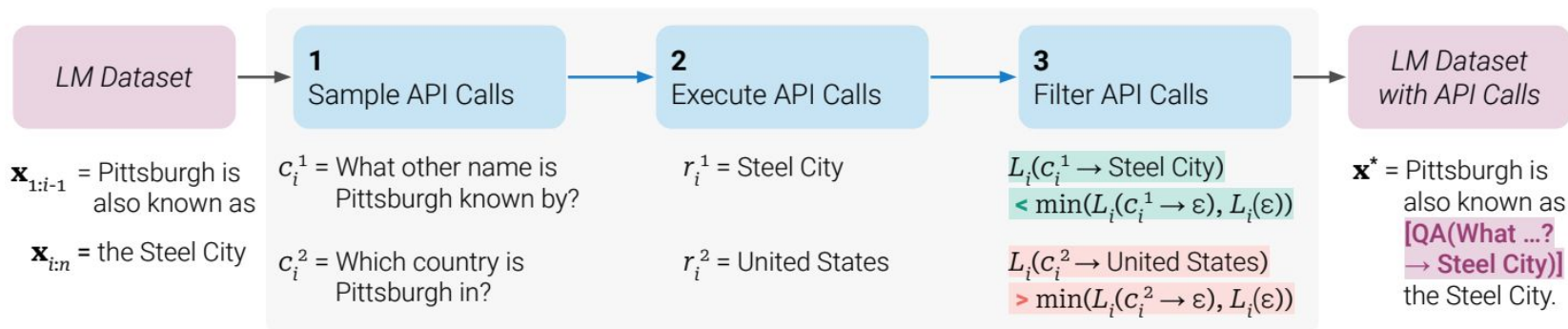
Learning from feedback: where the
supervision could come from either
the environment or humans.

Qin et al, 2023, *"Tool Learning with Foundation Models"*

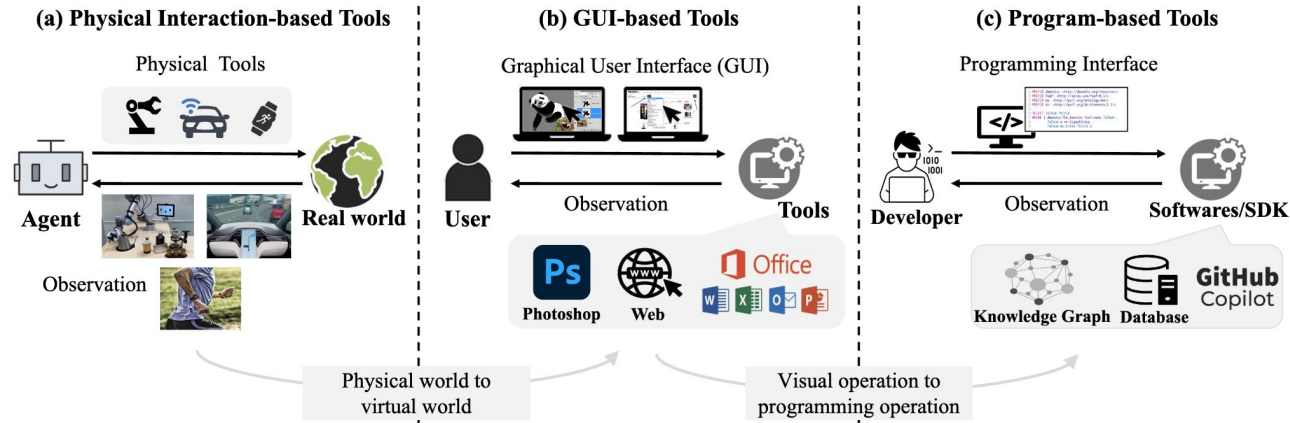
Toolformer: Supervised learning

Dataset Construction

1. Given an input text x , sample a position i and its corresponding API call candidates.
2. Execute API calls and filter out ones which do not reduce the loss over the next tokens.
3. All remaining API calls are interleaved with the original text, resulting in a new sequence.

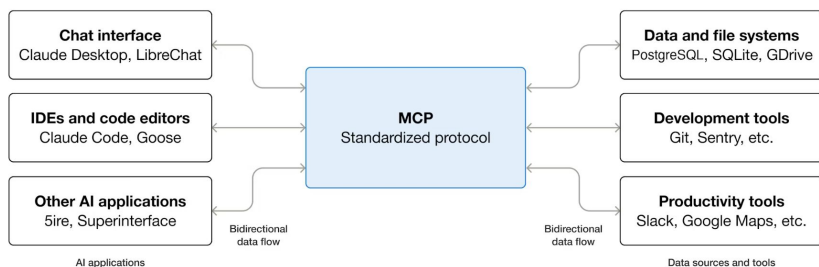


How to Connect Diverse Tools to LLM?



1. **Standard Tool-call Protocol:** structured format for calling and receiving
2. **Improve & Expand Model Capabilities:**
 - a. Structured output (JSON for example), instruction-following
 - b. Coding & SWE skills -- tools require code writing

Model Context Protocol (MCP)

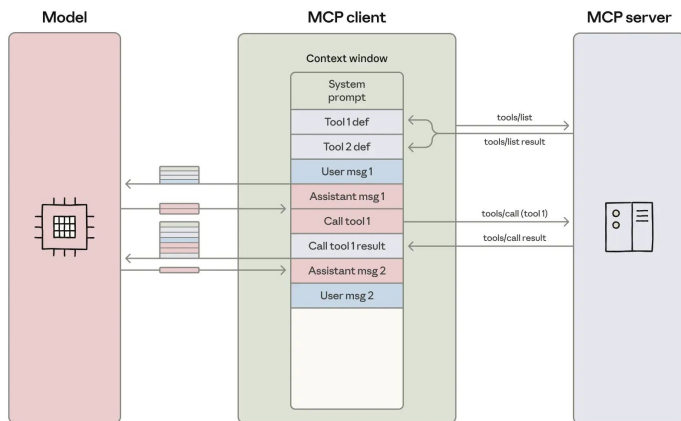


The USB-C port for tool connections for LLM tool calling



- ❖ **Developers:** reduces development time and complexity when building or integrating with a LLM tool calling pipeline.
- ❖ **AI Applications:** provides access to an ecosystem of data sources, tools and apps.
- ❖ **End-users:** results in more capable AI applications, can access your data and take actions on your behalf when necessary.

Model Context Protocol (MCP)



MCP defines 3 core primitives that servers expose:

- ❖ **Tools:** Executable functions that AI applications can invoke to perform actions
 - File operations, API calls, database queries
- ❖ **Resources:** Data sources that provide contextual information to AI applications
 - File contents, database records, API responses
- ❖ **Prompts:** Reusable templates that help structure interactions with language models
 - System prompts, few-shot examples

Model Context Protocol (MCP)

```
python

# weather_server.py
from mcp.server.fastmcp import FastMCP
import httpx

mcp = FastMCP("weather")

@mcp.tool()
async def get_forecast(city: str) -> str:
    """Get the current weather forecast for a city."""
    async with httpx.AsyncClient() as client:
        r = await client.get(f"https://wttr.in/{city}?format=3")
        return r.text

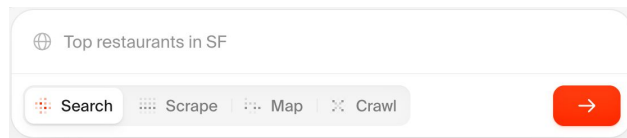
@mcp.resource("weather://cities")
def supported_cities() -> str:
    """List of cities this server knows about."""
    return "Paris, Tokyo, New York, Lausanne"

if __name__ == "__main__":
    mcp.run(transport="stdio")
```

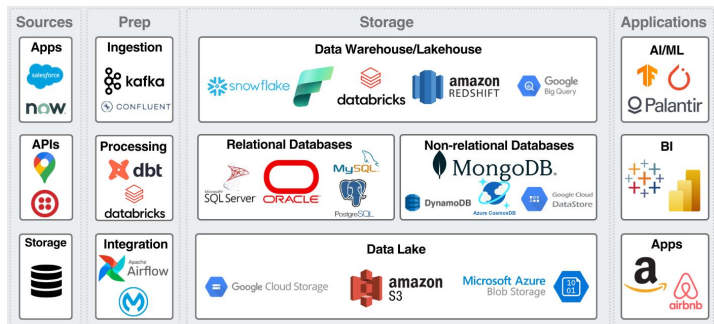
1. Client connects to the server and calls `tools/list` → discovers `get_forecast`.
2. The model decides to call `get_forecast(city="Lausanne")`.
3. Client forwards the call over MCP → server hits `wttr.in` → returns the result.
4. The model sees the tool result and writes a reply.

Common Modern LLM Toolboxes

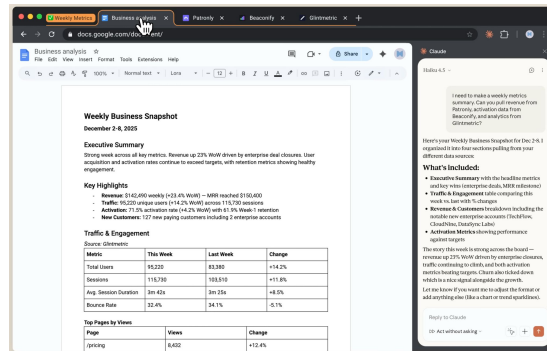
Tools for the Internet



Tools for Data/Cloud Applications



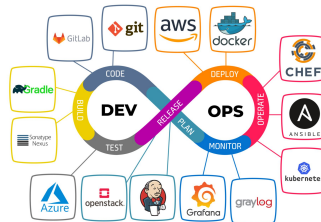
Tools for Browser-using



Terminal and FS Tools



Tools for DevOps



Tools for General Software Interaction

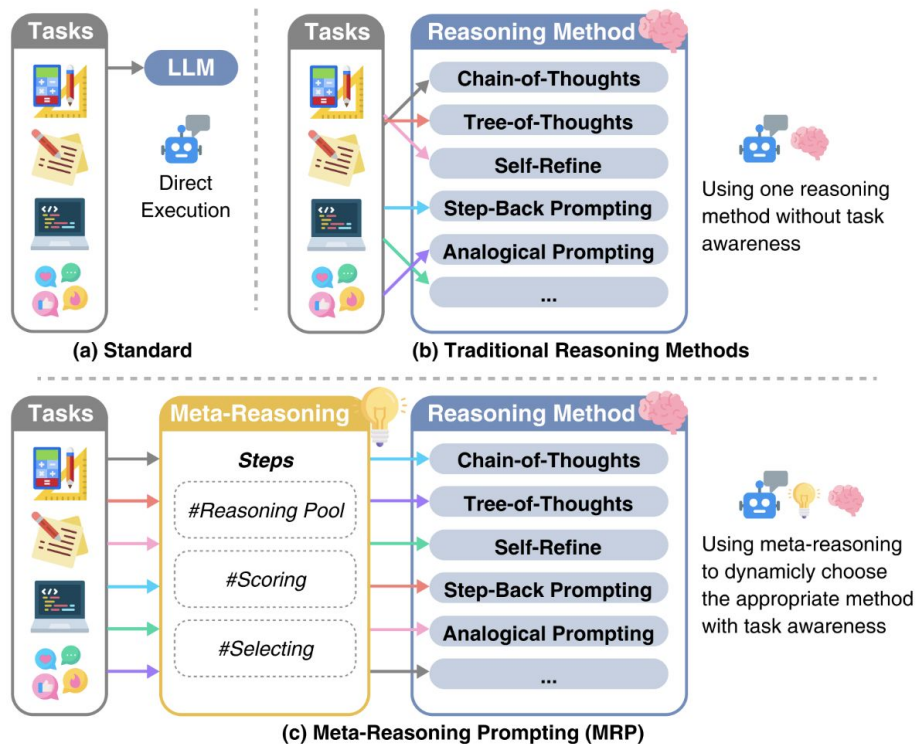


Model Context Protocol (MCP)

```
@mcp.prompt()
def explain_diff(diff: str) -> str:
    """Explain a git diff in plain English."""
    return (
        f"Explain what changed in this diff and why a reviewer might care. "
        f"Be concise.\n\n``diff\n{diff}\n``"
    )
```

- ❖ **Prompts are user-controlled:**
 - Templates that **users** can **explicitly invoke**, usually as a ***slash command*** in the client UI..
- ❖ **Output:** One or more messages that get inserted into the conversation as a **starting point**.

Cognitive Tools: Reasoning methods



From Tool-augmented LLMs to Agents

Tool-augmented LLM

- ❖ Model answers a question, optionally calls one or two tools, returns text

Agent - Autonomous Loop

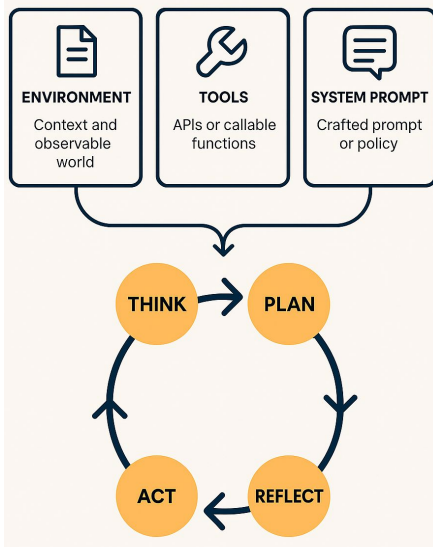
- ❖ Model pursues a goal across many steps, tools, and hours

Post-train a small language models to achieve improved performance on SWEbench-verified

1. **Research** on methods, datasets, evaluation, and metrics
2. **Implement** data, train, inference, and eval pipelines
3. Iterative **experiments** with train/eval loops
4. Final **report** writing with findings, reproducibility guide, and open-source artifacts

From Tool-Using LLMs to Agents

Building Effective AI Agents



Python

```
1 # ADK Pseudocode
2 # Step 1: Parse the PDF
3 parser = LlmAgent(
4     name="ParserAgent",
5     instruction="Parse raw PDF and extract text.",
6     tools=[PDFParser],
7     output_key="raw_text"
8 )
9
10 # Step 2: Extract structured data
11 extractor = LlmAgent(
12     name="ExtractorAgent",
13     instruction="Extract structured data from {raw_text}.",
14     tools=[RegexExtractor],
15     output_key="structured_data"
16 )
17
18 # Step 3: Summarize
19 summarizer = LlmAgent(
20     name="SummarizerAgent",
21     instruction="Generate summary from {structured_data}.",
22     tools=[SummaryEngine]
23 )
24
25 # Orchestrate the Assembly Line
26 pipeline = SequentialAgent(
27     name="PDFProcessingPipeline",
28     sub_agents=[parser, extractor, summarizer]
29 )
```

Agent Definition

- ❖ LLM (OpenAI, Claude, Kimi)
- ❖ Reasoning Effort (no, low, mid, high)
- ❖ System Prompt
- ❖ Available Tools
- ❖ Permission, Access, and Security Protocols
- ❖ Memory and Storage

ReAct: A Naive Agent Pattern

Question: Aside from the Apple Remote, what other device can control the program Apple Remote was originally designed to interact with?

Standard

Answer: iPod

Reason only

Thought: Let's think step by step. Apple Remote was originally designed to interact with Apple TV. Apple TV can be controlled by iPhone, iPad, and iPod Touch. So the answer is iPhone, iPad, and iPod Touch.

Answer: iPhone, iPad, iPod Touch

Act only

Act 1: Search[Apple Remote]
Obs 1: The Apple Remote is a remote control introduced...
Act 2: Search[Front Row]
Obs 2: Could not find [Front Row]. Similar: ...
Act 3: Search[Front Row (software)]
Obs 3: Front Row is a discontinued media center software ...
Act 4: Finish[yes]

ReAct

Thought 1: I need to search Apple Remote and find the program it was originally designed to interact with.

Act 1: Search[Apple Remote]

Obs 1: The Apple Remote is a remote control introduced in October 2005 by Apple ... originally designed to control the Front Row media center program ...

Thought 2: Apple Remote was originally designed to control the Front Row media center program. I need to search Front Row next and find what other device can control it.

Act 2: Search[Front Row]

Obs 2: Could not find [Front Row]. Similar: ['Front Row Seat to Earth', 'Front Row Motorsports', 'Front Row (software)', ...]

Thought 3: Front Row is not found. I need to search Front Row (software) .

Act 3: Search[Front Row (software)]

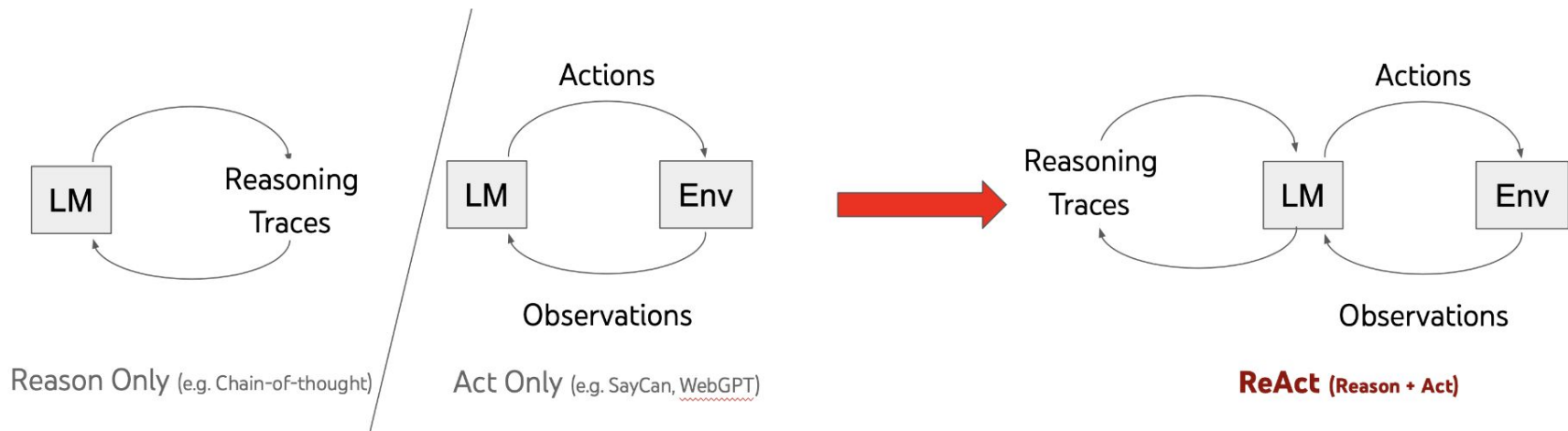
Obs 3: Front Row is a discontinued media center software ...

Thought 4: Front Row (software) is controlled by an Apple Remote or the keyboard function keys. So the answer is keyboard function keys.

Act 4: Finish[keyboard function keys]

ReAct: Interleave tool calls with thinking (planning + reflection)

ReAct: A Naive Agent Pattern



Lacks external
information

Lacks reasoning

- Breaks down complex task and plan for actions
- Execute actions using tools
- Reflect and plan for the next step

Agentic Harness

How do we deploy an agent in real-world systems?

- ❖ **Real-World is Messy:** complex, dynamic, risky, massive information, many use cases, massive user market, diverse software ecosystem

Robustness to a **complex & diverse software ecosystem**

Tasks are **long-horizon, and convoluted**

Task definitions are **vague and underspecified, require research effort**

Real-life tasks require an **iterative refinement and execution loop**

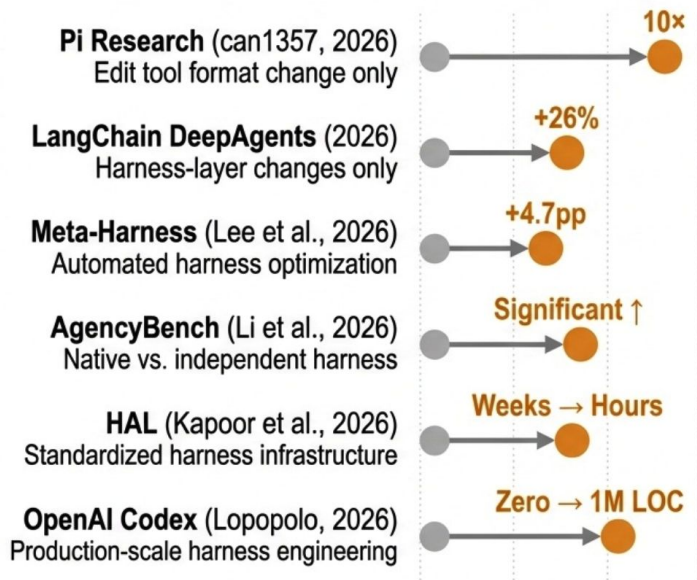
Production demands **reliability, scalability, not just capability**

Robustness to errors and potential security issues

Agentic Harness

Converging evidence: harness changes → large performance gains

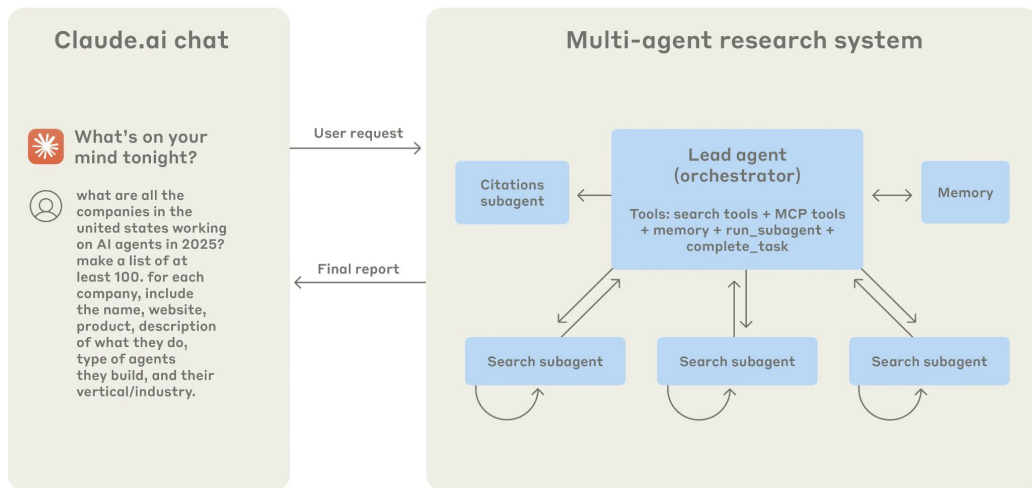
All improvements achieved with zero model changes



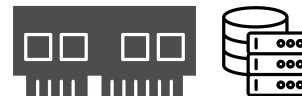
A **harness** is the scaffolding around the LLM that turns it into a production-level agent

- ❖ **Harness design** is key to performance at the frontier of agentic coding
- ❖ Scaling to **full-stack** software engineering
- ❖ **Co-evolution** between harness and LLMs

Agentic Harness: Building Blocks



Toolbox: search tools, MCP tools, DB tools, CLI tools, etc.



Memory: short-term context memory and long-term persistent memory.



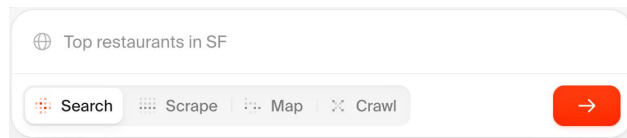
LLMs: general-purpose or specialized LLMs, pre-defined and optimized system prompts



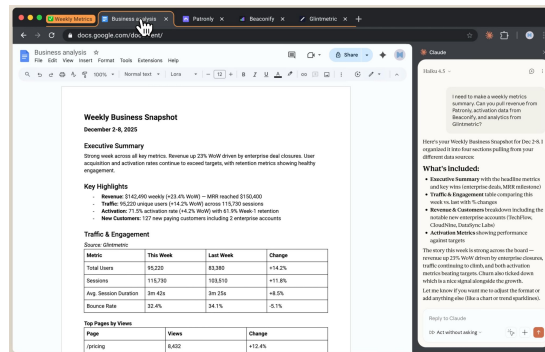
Environment: virtual filesystem, sandbox, browser

Common Modern LLM Toolboxes

Tools for the Internet



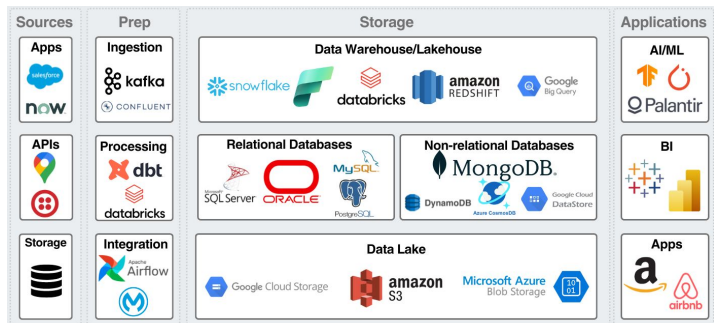
Tools for Browser-using



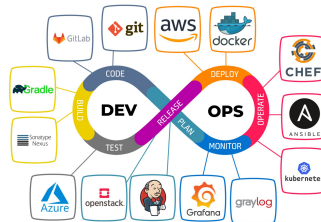
Terminal and FS Tools



Tools for Data/Cloud Applications



Tools for DevOps

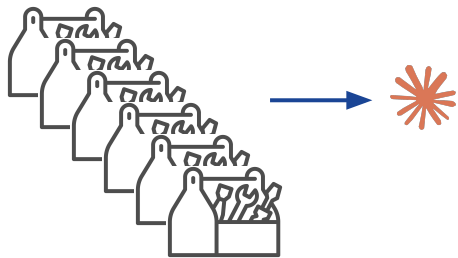


Tools for General Software Interaction



Agentic Harness: Efficient tool calling

What if you have 1M tools available?

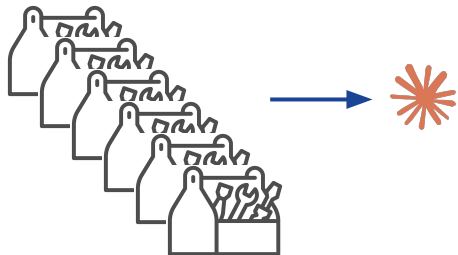


Agentic Harness: Efficient tool calling

What if you have 1M tools available?

Tool-search tool:

- ❖ Present tools as code on a filesystem
- ❖ Agent can read only the definitions it needs, tool loading on demand

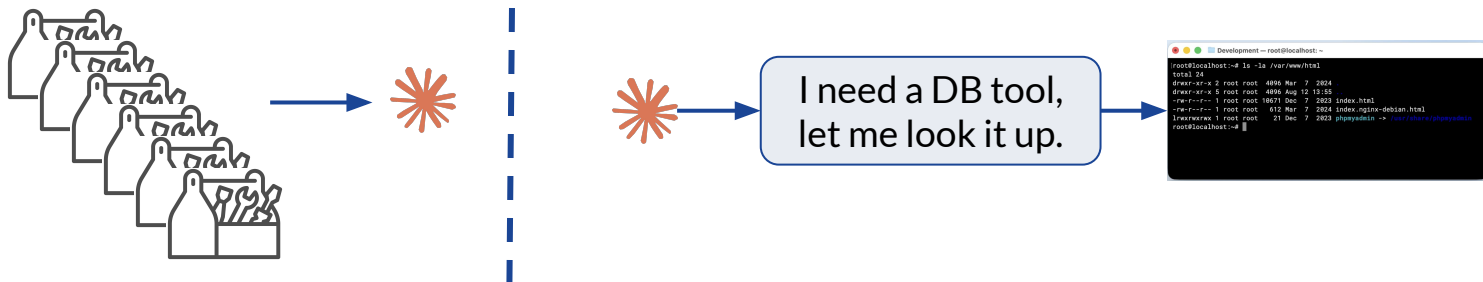


Agentic Harness: Efficient tool calling

What if you have 1M tools available?

Tool-search tool:

- ❖ Present tools as code on a filesystem
- ❖ Agent can read only the definitions it needs, tool loading on demand

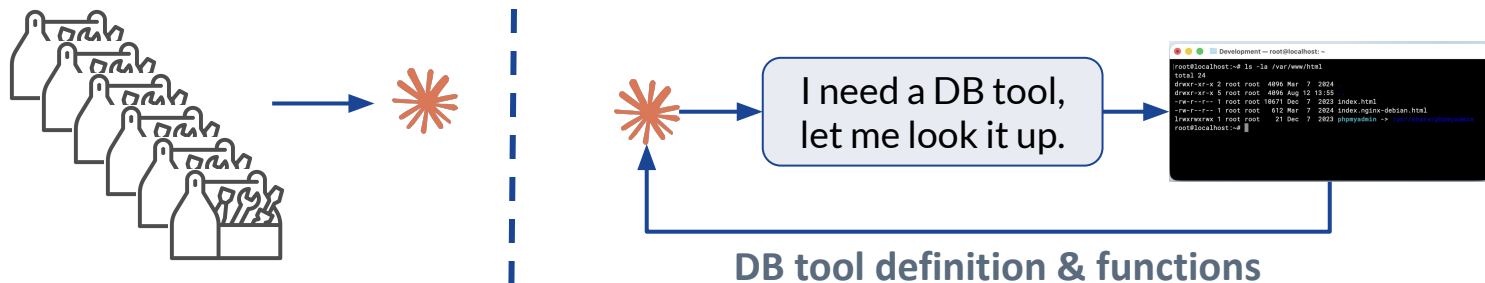


Agentic Harness: Efficient tool calling

What if you have 1M tools available?

Tool-search tool:

- ❖ Present tools as code on a filesystem
- ❖ Agent can read only the definitions it needs, tool loading on demand



Agentic Harness: Secure tool calling

Purpose of a Sandbox:

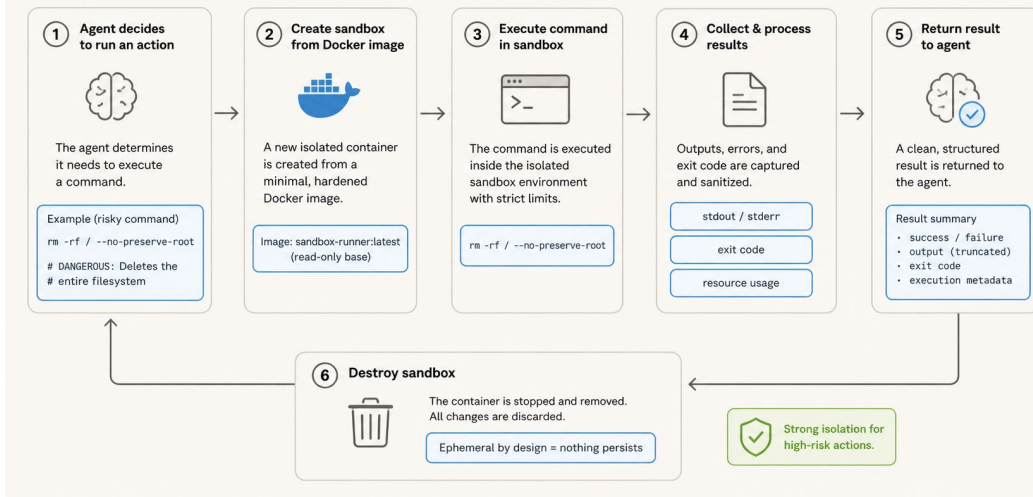
1. Contains untrusted code
2. Secure execution of risky actions
3. Limited exposure and damage
4. Reproducible actions for auditing

Desired properties:

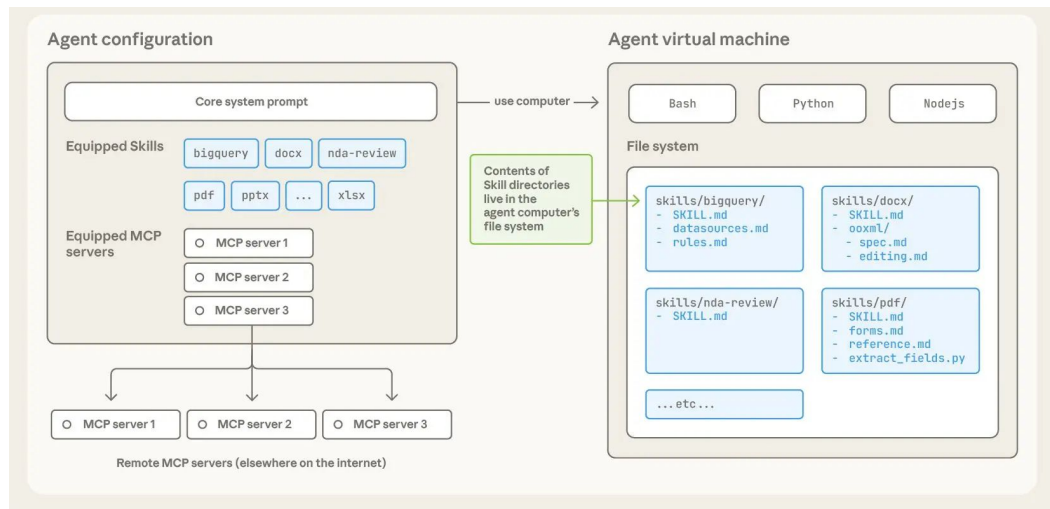
1. Parallel execution
2. Efficiency, low latency
3. State logging for auditability



LLM Agent Harness: Execute Code / Linux Commands in a Sandbox



Agentic Harness: Tool calling with many layers

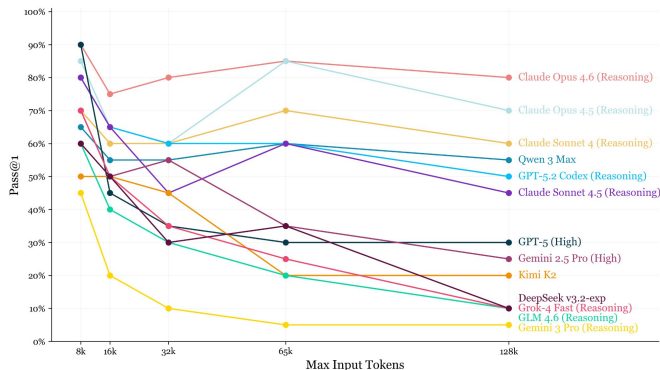


1. MCP Servers
2. Filesystems
3. Virtual machine terminals
4. Virtual machine coding
5. Browser (Claude-in-Chrome)
6. Markdown skills + scripts
7. System prompts
8. Subagents

Agentic Harness: Context Engineering

LongCodeEdit - Hard

Long Context Code Editing



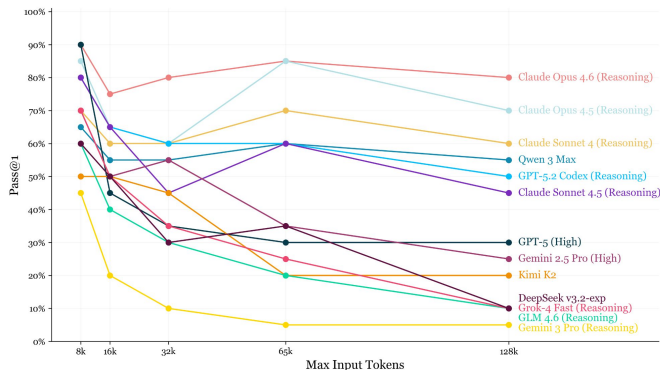
Context Rot: Performance degradation when the #tokens increase in the context.

- ❖ **Context window** has got much larger in recent LLMs
 - Position embedding with **1K-4K** to **128K-1M**.
 - **Long-context reasoning** is still a challenge for many LLMs

Agentic Harness: Context Engineering

LongCodeEdit - Hard

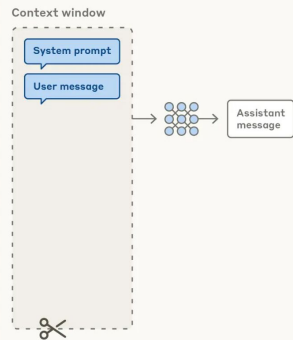
Long Context Code Editing



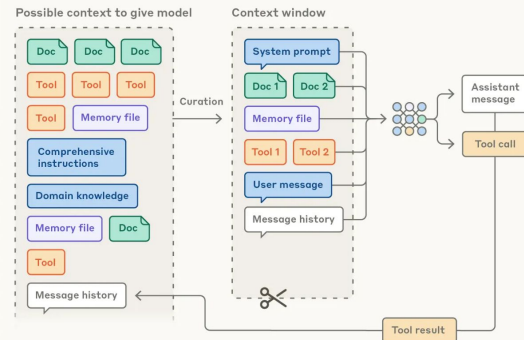
Context Rot: Performance degradation when the #tokens increase in the context.

Prompt engineering vs. context engineering

Prompt engineering for single turn queries



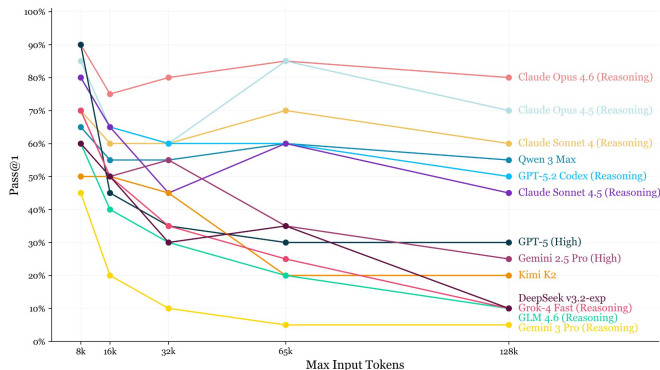
Context engineering for agents



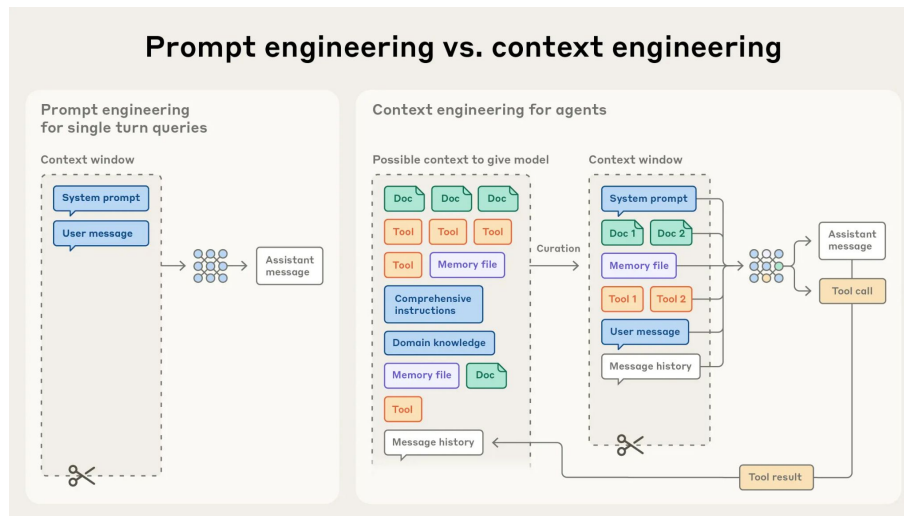
Agentic Harness: Context Engineering

LongCodeEdit - Hard

Long Context Code Editing



Context Rot: Performance degradation when the #tokens increase in the context.

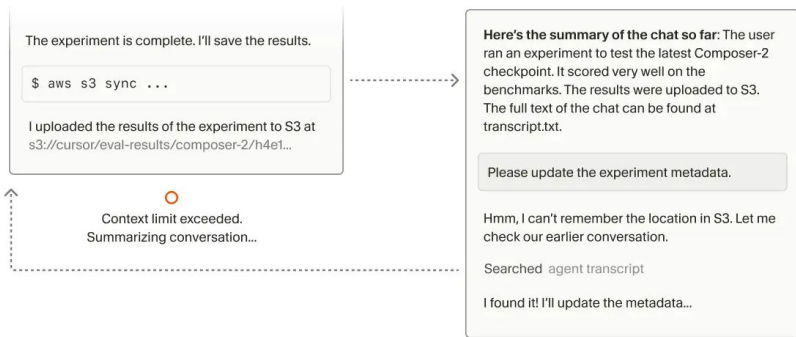


Core Idea: Provide only the **right information** to the LLM in its **context window**

- ❖ **Prompt Engineering:** writing and organizing LLM instructions for optimal outcomes.
- ❖ **Context Engineering:** curating and maintaining the optimal set of tokens (information) during LLM inference

Agentic Harness: Context Engineering

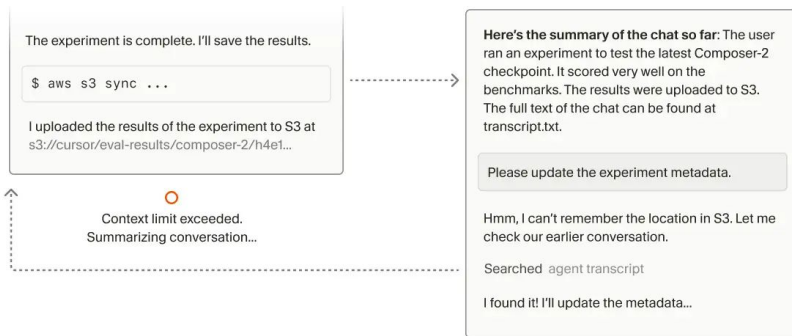
Referencing past chats improves consistency
over long-running conversations



FileSystem-as-Memory Pattern

Agentic Harness: Context Engineering

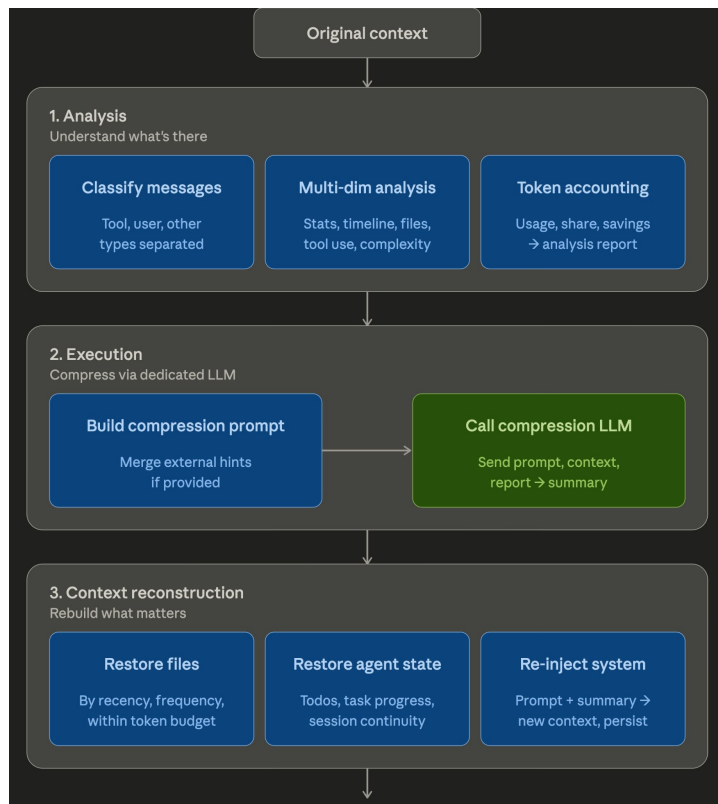
Referencing past chats improves consistency
over long-running conversations



FileSystem-as-Memory Pattern

- ❖ **Offload** large tool results or documents to the **filesystem/DB**
- ❖ Replace the **original tokens (ex. 32K)** with a short **reference (ex. 128)**
- ❖ Load back the contents **on demand** through terminal commands (grep, glob, etc.)
 - **Agentic Search:** find the right information through code actions

Agentic Harness: Context Engineering



Context Compression

- ❖ Triggered when the context is **larger** than a **threshold** (92% of the window)
- ❖ Preventing **context bloat** or **exceeding** the limit
- ❖ Preserve important messages, fine-grained compression

Case Study: Huggingface ML-Intern

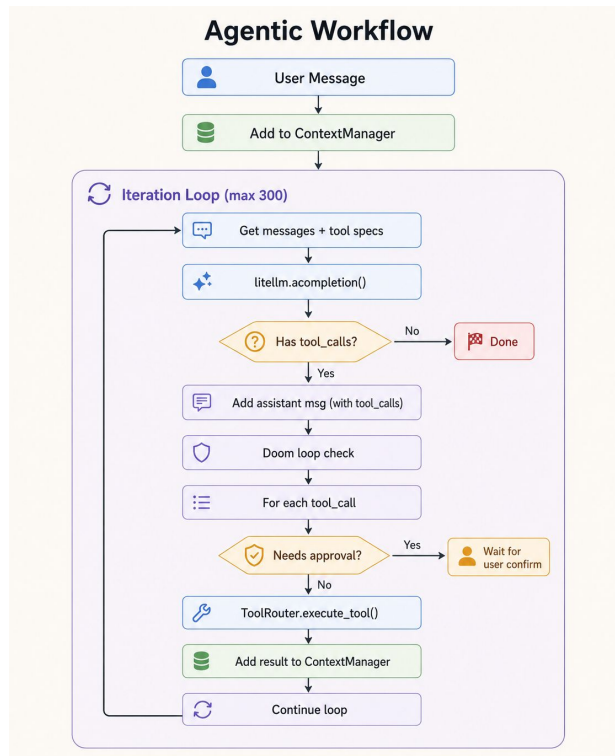
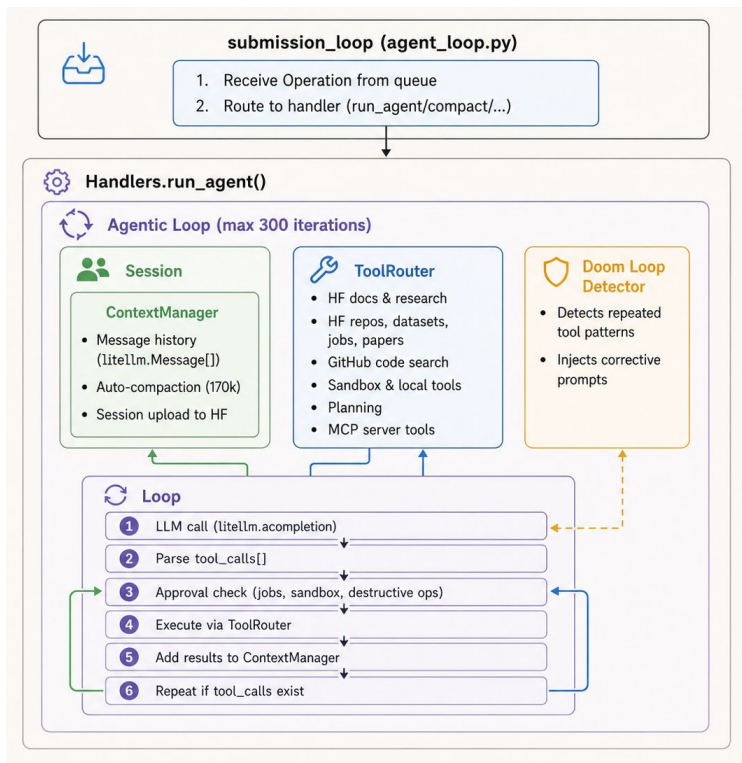
Task: Post-train a small LLM to achieve improved performance on SWEbench-verified

How do we design an agentic harness to automatically solve this task?

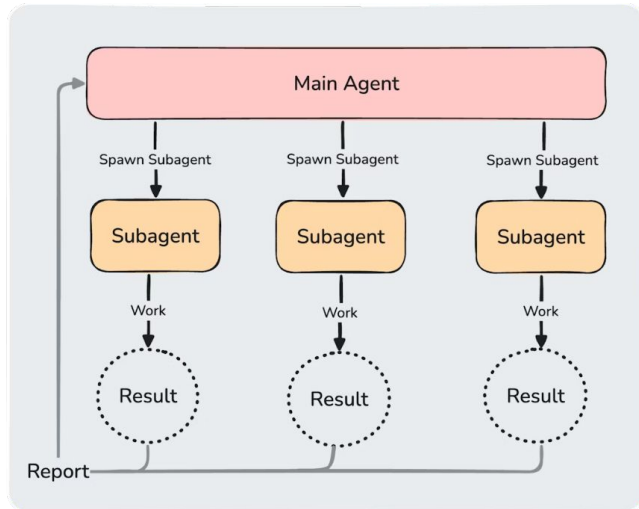
<https://github.com/huggingface/ml-intern>

Case Study: Huggingface ML-Intern

Post-train a small LLM to achieve improved performance on SWEbench-verified



Agentic Harness: Multi-agent Coordination



What it is: Multiple agents working together on a single goal, each with its own context, tools, and role.

Why we need it:

- ❖ A **single agent** has **one** context window, reasoning budget, role, and sequential trajectory.
- ❖ Complex tasks often exceed all four.
- ❖ Multi-agent systems can **scale beyond the limits** of a single instance, in **capability, throughput, and reliability**.

Advantages:

1. Specialization & Delegation
2. Parallelism & Large-context offloading
3. Compositional execution
4. Consolidated reasoning effort & divide-and-conquer on hard problems

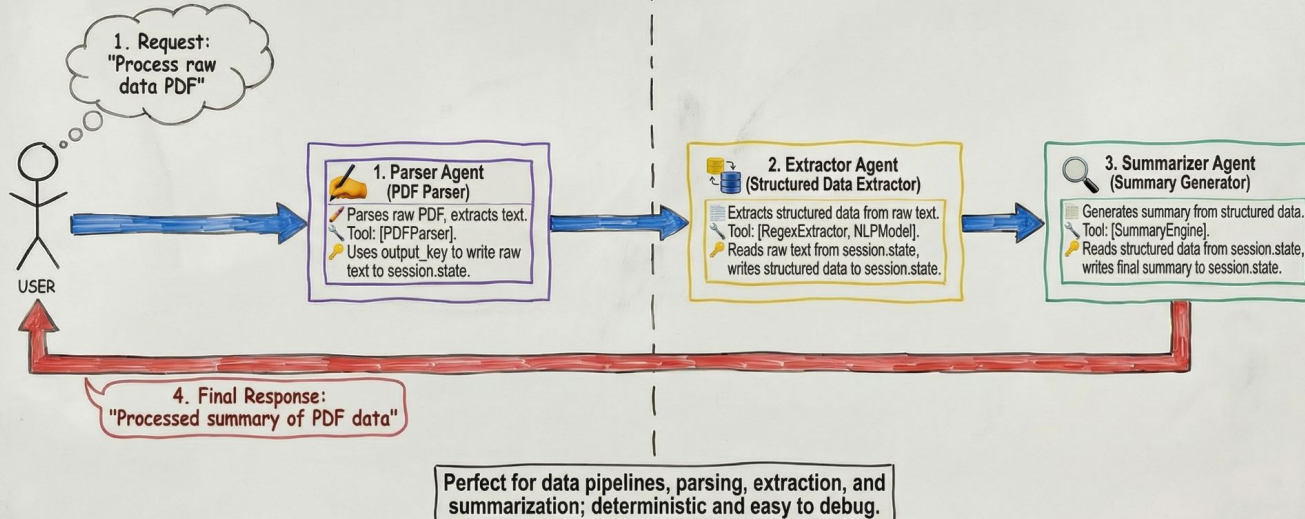
Agentic Harness: Multi-agent Coordination

Sequential Pipeline Agent Pattern

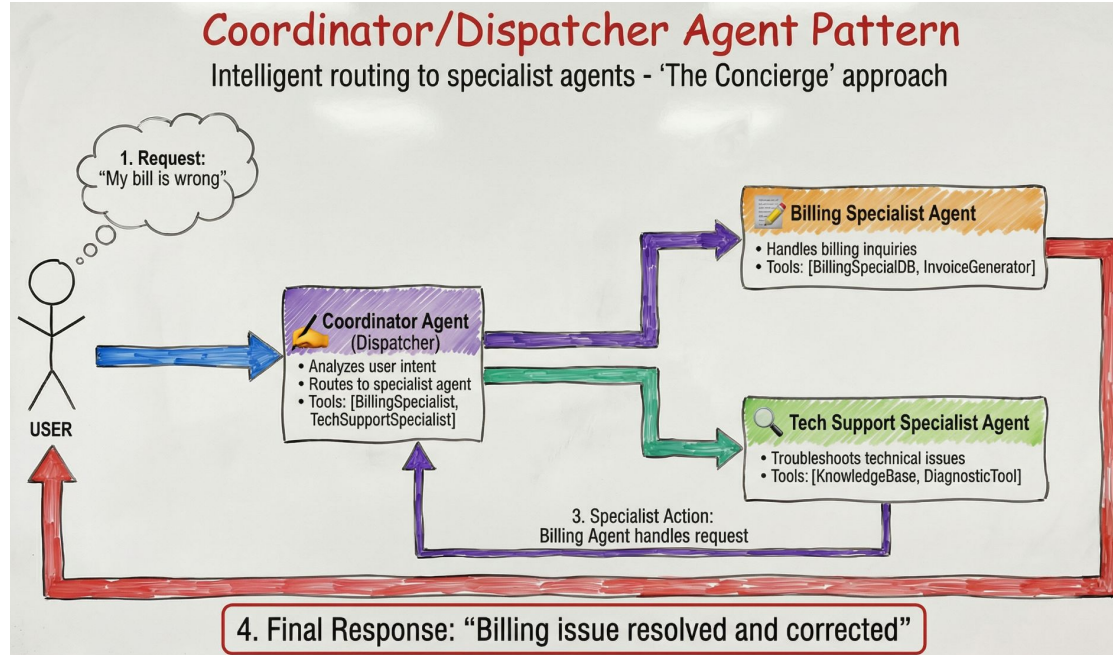
Linear process for data processing, like an assembly line - The Assembly Line approach

ARCHITECTURE

AGENT PIPELINE



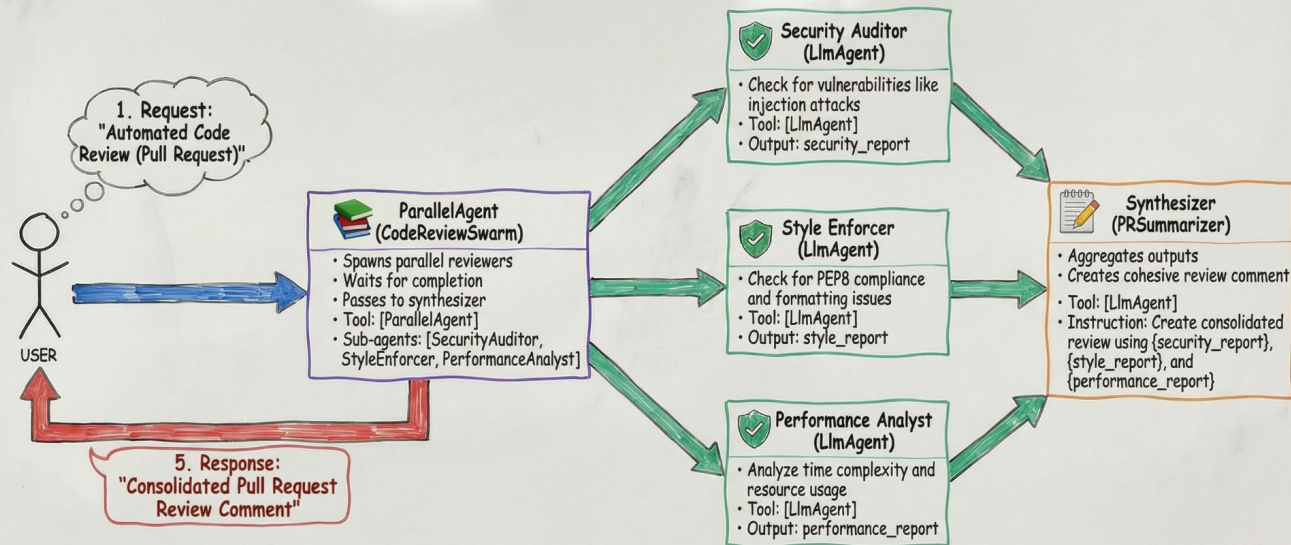
Agentic Harness: Multi-agent Coordination



Agentic Harness: Multi-agent Coordination

Parallel Fan-Out Agent Pattern

Concurrent execution for speed and diverse perspectives - "The Swarm" approach



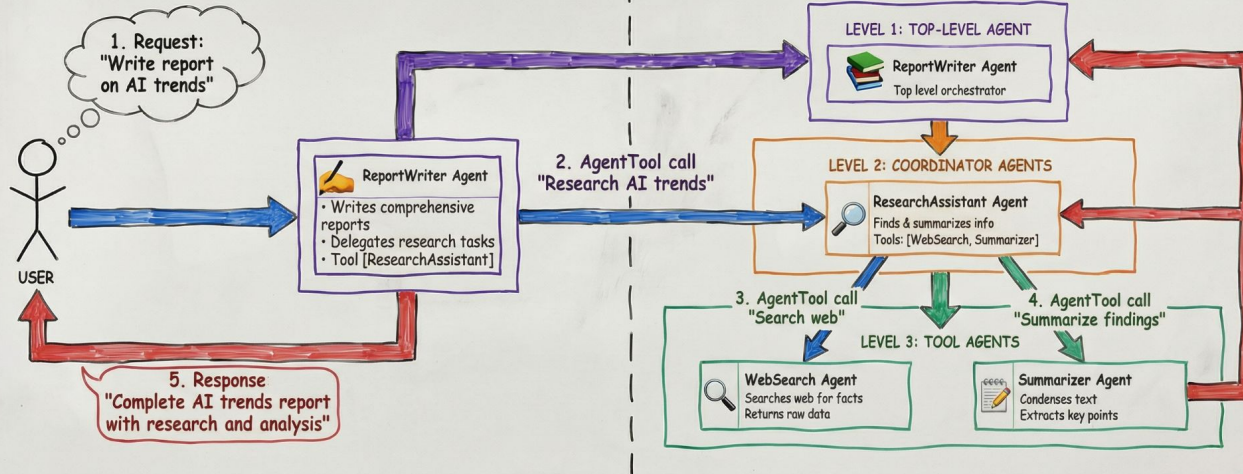
Agentic Harness: Multi-agent Coordination

Hierarchical Task Decomposition Agent Pattern

Agents within agents within agents - The Russian Doll approach

ARCHITECTURE

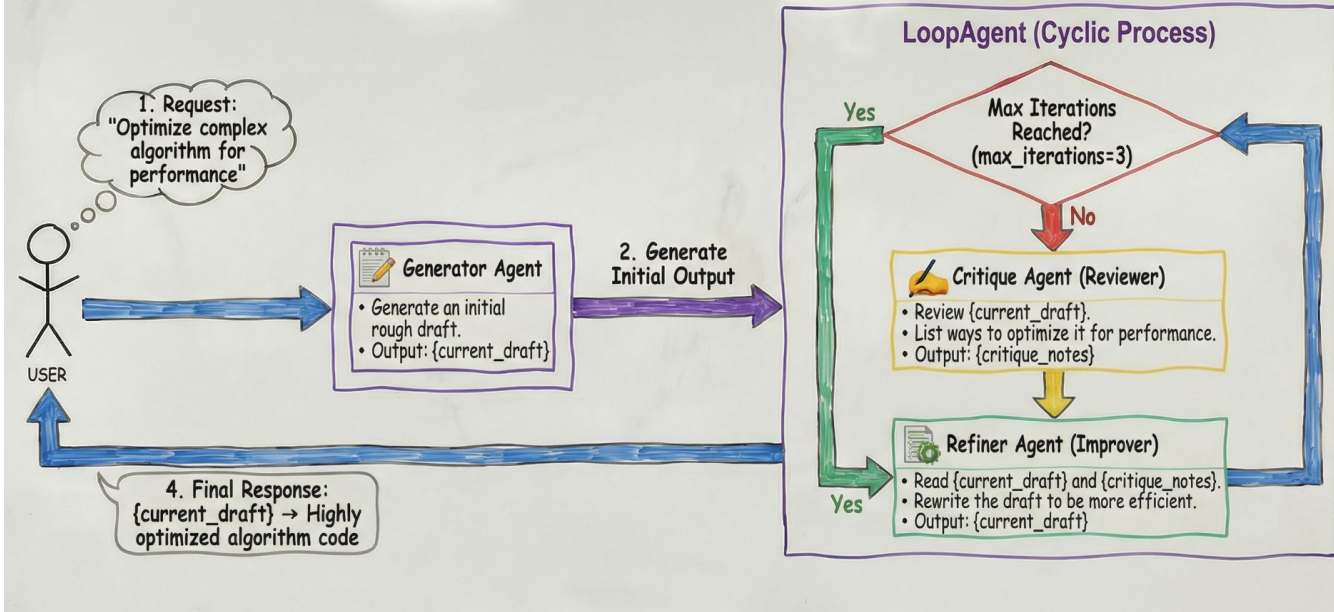
AGENT HIERARCHY



Agentic Harness: Multi-agent Coordination

Iterative Refinement Agent Pattern

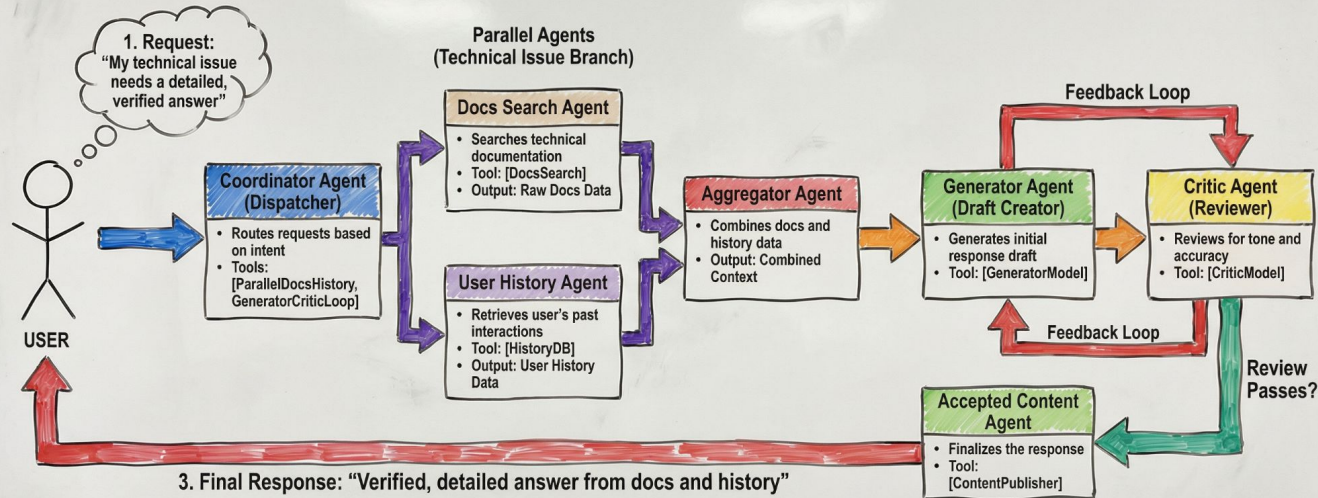
Cyclic process of generating, critiquing, and refining - "The Sculptor" approach.



Agentic Harness: Multi-agent Coordination

Composite Agent Pattern

Real-world enterprise applications - 'The Mix-and-Match' approach



Perfect for complex, multi-step workflows requiring parallel processing, review loops, and specialized handling.

Agentic Harness: Common Systems



openai-agents-python



Tongyi DeepResearch



claude-agent-sdk-python



Agent Development Kit (ADK)

agno
AgentOS



LangChain

OpenClaw

THE AI THAT ACTUALLY DOES THINGS.

opencode

Ask anything... "Fix broken tests"

Build Claude Opus 4.5 OpenCode Zen

tab switch agent ctrl+p commands

Evaluating LLM Agents

Evaluating Agent is **Difficult**

- ❖ **Task completion:** Did the agent achieve the intended goal?
- ❖ **Reasoning & planning:** Quality of decomposition, tool selection, and refinement.
- ❖ **Tool use & grounding:** Precision of API/tool calls and faithfulness to tool results.
- ❖ **Efficiency & cost:** Number of steps, tokens, latency, and dollar cost per task.
- ❖ **Safety & alignment:** Refusal of harmful actions, robustness to prompt injection, side-effect control.

Evaluation Dimensions to Consider

- ❖ **Outcome vs. process:** final answer correctness vs. trajectory/step-level quality.
- ❖ **Static vs. interactive:** fixed benchmarks vs. live environments and multi-turn rollouts.
- ❖ **Single-agent vs. multi-agent:** individual capability vs. coordination and communication.
- ❖ **Generalization:** in-distribution tasks vs. unseen tools, domains, and longer horizons.

Evaluating LLM Agents

Current Practice: Heavily in Coding, SWE, & GUI domains

- ❖ **Coding + SWE:**
 - [SWE-bench](#), [TerminalBench](#) 1&2, [KernelBench](#), [PostTrainBench](#), [tau-bench](#)
- ❖ **GUI interactions:**
 - [WebArena](#), [VisualWebArena](#), [OSWorld](#).
- ❖ **General purpose:** [AgentBench](#), [GAIA](#)
- ❖ **Sandboxed environments and containerized rollouts for reproducibility.**
- ❖ **Hybrid evaluation:**
 - Automated checks + LLM judges + targeted human review.
- ❖ **Observability tooling:**
 - LangSmith, Braintrust, Phoenix
 - Traces, replay, and regression tests
- ❖ **Open challenges:**
 - Contamination, reward hacking, judge bias, and weak long-horizon evaluation.

Reading List

1. <https://www.anthropic.com/engineering/code-execution-with-mcp>
2. <https://www.anthropic.com/engineering/equipping-agents-for-the-real-world-with-agent-skills>
3. <https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents>
4. <https://www.anthropic.com/engineering/harness-design-long-running-apps>
5. <https://docs.langchain.com/oss/python/deepagents/overview>